Institut Teknologi Bandung

Directorate of Sustainable Professional Education

CHAPTER 19

The Future of AI

Where deep learning falls short, why scale does not fix it, and what a system would need in order to handle a situation nobody anticipated.

Duration 2.5 hours (2 sessions)

Instructor Prof. Bambang Riyanto Trilaksono · Rahman Indra Kesuma, S.Kom., M.Cs.

Source Chollet & Watson, Deep Learning with Python 3e — chapter 19

LEARNING OUTCOMES

Session Objectives

By the end of this session, participants can:

1. State the **four structural limitations** of deep learning, and give a concrete failure for each.
2. Explain why an LLM's success on a problem tracks **familiarity, not complexity**.
3. Explain what **prompt engineering actually is** in terms of the interpolative database, and what its necessity implies.
4. Argue why **scaling laws do not answer** any of these limitations.
5. Distinguish **local, broad, and extreme generalization**, and place current systems on that spectrum.
6. Define intelligence as an **efficiency ratio** between information available and future operating area.
7. Explain what **ARC-AGI** measures, why it resisted a 50,000× scale-up, and what test-time adaptation changed in 2024.
8. Compare the **two poles of abstraction** and say why a complete system needs both.

BOOK

Chapter 19 — full text

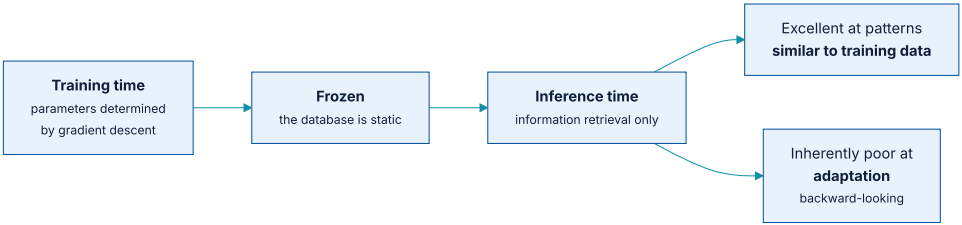
01

The limitations of deep learning

To use a tool well, be aware of its limitations, not just its strengths.

SECTION 19.1.1

A parametric curve is a database, and a static one



Chapter 15 called it an interpolative database. The word that matters here is *static*.

The only thing you can do with a static database is information retrieval. At inference time you had better hope the situations the model faces are inside the training distribution.

SECTION 19.1.1

The leopard-print sofa

A model trained on ImageNet will classify a **leopard-print sofa** as an actual leopard. Sofas were not part of its training data.

This applies equally to the largest generative models. It is frequently claimed that LLMs perform **in-context learning** — picking up new skills from a few examples. There is overwhelming evidence that what they are actually doing is **fetching vector functions memorized during training** and reapplying them.

By learning next-token prediction across a web-sized dataset, an LLM has collected millions of potentially useful mini text-processing programs, and can be prompted into reusing them. **Show it something with no direct equivalent in its training data, and it is helpless.**

Familiarity, not complexity

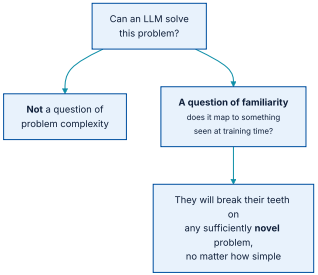


Figure 19.1's puzzle is easy for you and unsolved by every state-of-the-art model.

This is the single most useful reframing in the chapter for anyone deciding where to deploy an LLM. **The question is never "is this task hard?" It is "is this task familiar?"**

Ten kilos of steel, and Monty Hall

> *"What's heavier, **10 kilos** of steel or one kilo of feathers?" — For months after ChatGPT's release, the answer was that **they weigh the same**.*
>
> Section 19.1.1

The question *"one kilo of steel or one kilo of feathers?"* appears many times online as a trick question, with that exact answer. The model repeated the memorized answer **without attending to the actual numbers** or what the query meant.

The same happens with variations of the **Monty Hall problem**: the canonical answer comes out regardless of whether it makes sense in the altered context.

Twenty-five thousand people playing whack-a-mole

25,000+
people employed full time providing LLM training data

one at a time
how failing prompts get patched

LLM maintenance is a constant game of whack-a-mole where failing prompts are patched individually, **without addressing the underlying issue**. **Even already-patched prompts will fail again if you make small changes to them.**

Adversarial examples: a panda plus a gibbon gradient

Take a picture of a panda, add a gibbon class gradient, and the network classifies it as a gibbon. The change is **imperceptible to a human**.

> This evidences both the brittleness of these models and the **deep difference between their input-to-output mapping and human perception**. The two are not approximations of each other.

Alice has N brothers and M sisters

> *"Alice has N brothers and she also has M sisters. How many sisters does Alice's brother have?"*
>
> The Alice in Wonderland riddle — Nezhurina et al., arXiv 2406.02061

The answer is **M + 1** — Alice's sisters plus Alice herself. With values commonly found in online instances of the riddle (N = 3, M = 2) an LLM generally answers correctly. **Tweak the values and you quickly get wrong answers.**

Changing place names, people's names in a paragraph, or variable names in a block of code can **significantly degrade** performance. None of those changes alters the problem.

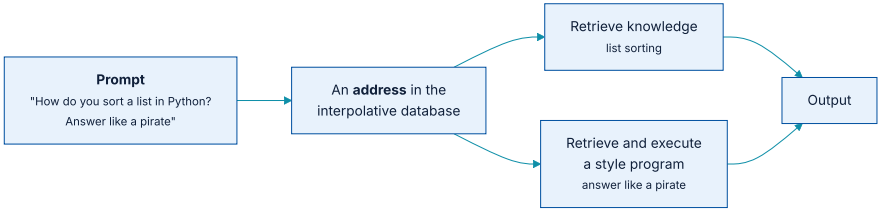
Two framings of the same fact

The optimistic framing

Your models are better than you know! You just need to use them right. Adding **"Please think step by step"** can significantly boost performance on reasoning tasks.

The negative framing

For any query that seems to work, there is a range of **minor changes that can tank performance**. To what extent does a model understand something if a rewording breaks that understanding?



A prompt is an address, not an instruction.

SECTION 19.1.2

Prompt engineering is search through latent space

There are thousands of variations, each giving a similar yet slightly different answer. **There is no a priori reason for your first naive prompt to be optimal.**

> Prompt engineering is trial-and-error search for the lookup query that performs best. **It is no different from trying different keywords in a Google search.** If LLMs actually understood what you asked, **there would be no need for this search process at all**

Memorized programs do not generalize

This is especially true of programs encoding **discrete logic**. Train a Transformer on hundreds of thousands of digit-addition pairs like `"4 3 5 7 + 8 9 3 6"` and you reach very high accuracy — **very high, but not 100%**.

~70%
state-of-the-art LLM accuracy on digit addition

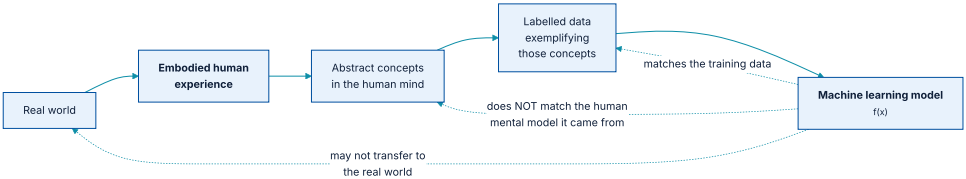
which digits
accuracy depends on it — common digits do better

Unless, that is, the model was explicitly hardcoded to execute `4357 + 8936` in Python.

Most programs cannot be expressed as a deep learning model

A deep learning model can be interpreted as a kind of program. But inversely, **most programs cannot be expressed as deep learning models.** For most tasks either no network of reasonable size solves it, or one exists but is not learnable — the transform is too complex, or the data does not exist.

A dim image in a mirror



What the model learns matches the training data — not the mental model the data came from.

The models you create will take any shortcut available to fit their training data.

Theory of mind, applied where it does not belong

Applied to deep learning, this leads us to believe a model that uses language **understands** the word sequences it generates the way we do. Then we are surprised when a slight departure from the training patterns produces something absurd.

> Never fall into the trap of believing neural networks understand the task they perform. **They were trained on a different, far narrower task than the one you wanted to teach them**: mapping training inputs to training targets, point by point.

02

Scale isn’t all you need

Four or five orders of magnitude larger, and the same problems.

SECTION 19.2

The prevailing narrative, and what it missed

Proponents pointed to **scaling laws**: an empirical relationship between model and dataset size and performance on specific tasks, suggesting that size reliably and predictably buys performance.

> The key thing scaling-law enthusiasts miss is that the benchmarks measuring "performance" are effectively **memorization tests** — the kind we give university students. LLMs do well by memorizing the answers, and **cramming more questions and answers in improves the score accordingly.**

Seven years, four orders of magnitude, same problems

Scaling has produced **no progress** on inability to adapt to novelty, oversensitivity to phrasing, or the inability to infer generalizable programs — because these issues are inherent to **curve fitting**.

Chollet began pointing out these problems in **2017**. We are still struggling with them, with models four or five orders of magnitude larger and more knowledgeable.

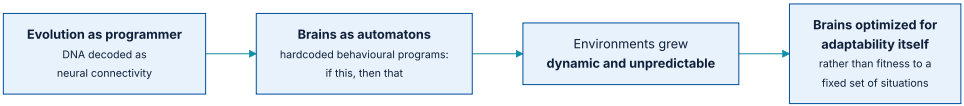
> **We have made no progress because the models are still the same.** They have been the same for over seven years: parametric curves fitted to a dataset by gradient descent, using the Transformer architecture.

Three things stacking more layers will not solve

1. Models are **limited to interpolative programs memorized at training time**. They cannot, on their own, synthesize brand-new programs at inference time to adapt to substantially novel situations.
2. Even within known situations, those interpolative programs suffer **generalization issues** — oversensitivity to phrasing and confounder features.
3. Models are **limited in what they can represent**. Most programs you would wish to learn cannot be expressed as a continuous geometric morphing of a data manifold — algorithmic reasoning tasks in particular.

SECTION 19.2.1

Why brains appeared, and what they were at first



Brains appeared more than half a billion years ago as a way to store and execute behavioural programs.

Because the source code was **DNA**, decoded as neural connectivity, evolution could suddenly search over behaviour space in a largely unbounded way. Eyes, ears, mandibles, four legs, twenty-four legs — **brains handle any modality you throw at them**.

SECTION 19.2.2

Automatons and intelligent agents

An automaton

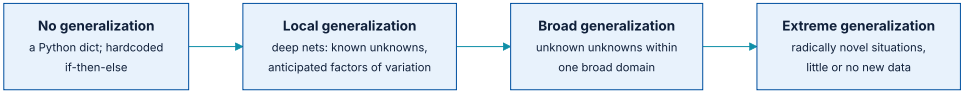
Static, crafted to accomplish specific things in a specific context. Expose it to something outside what it was programmed for — by hand, by evolution, or by fitting on a dataset — and **it fails**.

An intelligent agent

Adapts **on the fly** to novel, unexpected situations, using fluid intelligence to find a way forward.

How do you tell a student who memorized three years of past exam questions from one who understands the subject?
You give them a brand-new problem.

The spectrum of generalization



Deep nets generalize to *known unknowns* — factors of variation anticipated during development and featured in the training data.

Deep nets generalize **by interpolation on a manifold**, so any factor of variation in the input space must be captured by the manifold they learn. That is why basic **data augmentation** helps so much — and why nothing helps when the variation was never anticipated.

Landing a rocket, and crossing a road

The rocket

A deep net needs **tens of thousands or millions of launch trials** — a dense sampling of the input space. Humans build a physical model, *rocket science*, and land it in one or a few tries.

The road

A net controlling a body would have to **die many thousands of times** to infer that cars are dangerous. Dropped into a new city, it would relearn most of what it knows.

Humans learn safe behaviours **without dying even once** — thanks to the power of abstract modelling of novel situations.

SECTION 19.2.3

What intelligence is for

But as organism and environmental complexity rose, situations became dynamic and unpredictable. **A day in your life is unlike any day you have experienced, and unlike any day experienced by any of your evolutionary ancestors.**

> *Intelligence is the ability to* ***efficiently use the information at your disposal to produce successful behaviour in the face of an uncertain, ever-changing future.***
>
> Section 19.2.3

SECTION 19.2.4

Seventy years of AI, on the same spectrum

ERA	SYSTEM	WHERE ON THE SPECTRUM
1960s-70s	ELIZA; SHRDLU manipulating objects from language commands	**Pure automatons**
1990s-2000s	Machine learning systems handling some uncertainty	**Local generalization**
2010s	Deep learning: larger datasets, more expressive models	**Local generalization, expanded**
Today	Self-driving cars; a robot that could make coffee in a random kitchen	**Toward broad generalization**

The last row is the **Woz test of intelligence** — enter a random kitchen and make a cup of coffee. Visible progress is being made, by **combining deep learning with painstakingly handcrafted abstract models of the world**.

Artificial cognition, not artificial intelligence

> *The "intelligence" label in "artificial intelligence" has been a **category error**. It would be more accurate to call our field artificial cognition, with cognitive automation and artificial intelligence being two nearly independent subfields within it.*
>
> Section 19.2.4

In that subdivision, AI would be **a greenfield where almost everything remains to be discovered**.

> This is not meant to diminish deep learning. Cognitive automation is incredibly useful, and automating tasks from exposure to data alone is **far more practical and versatile than explicit programming**. **Doing it well is a game changer for essentially every industry.**

03

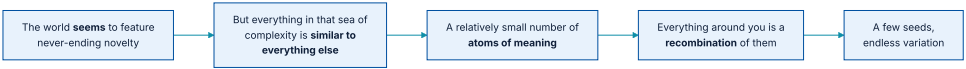
How to build intelligence

Abstraction acquisition, on-the-fly recombination, and a benchmark designed to resist memorization.

SECTION 19.3.1

The kaleidoscope hypothesis

A person from the 17th century seeing a jet plane might describe *a large, loud metal bird that doesn't flap its wings*. A car is a **horseless carriage**. Electricity is like water in a pipe; spacetime is like a rubber sheet distorted by heavy objects.



A few beads of coloured glass, reflected by mirrors, produce endless patterns.

SECTION 19.3.1

The analogies are not only in our heads

And **physical reality itself is full of isomorphisms**. Electromagnetism is analogous to gravity. Animals are structurally similar because of shared origins. Silica crystals resemble ice crystals.

> The number of unique **atoms of meaning** needed to describe the universe you live in is relatively small, and everything around you is a **recombination of them**

SECTION 19.3.2

Intelligence in two parts

Extract the beads; recombine them on the fly into a model of the situation in front of you.

> **The emphasis on efficiency is crucial.** If you need hundreds of thousands of hours of practice to acquire a skill, you are not very intelligent. If you need to enumerate every possible move on a chessboard to find the best one, you are not very intelligent.

Deep learning has one half, and it is the wrong half

No on-the-fly recombination

Models do a decent job of **acquiring** abstractions at training time via gradient descent, but have **zero ability to recombine** what they know at test time. They are a static database limited to retrieval — missing half the picture, arguably the most important half.

Terribly inefficient

Gradient descent requires vast amounts of data to distill neat abstractions — **many orders of magnitude more than humans**.

You get what you measure, and nothing else

The shortcut rule: optimize one success metric and you will achieve your goal — at the expense of everything in the system your metric did not cover.

In 2009 Netflix ran a $1 million challenge for the best movie recommendation score. **It never used the winning system** — far too complex and compute intensive. The winners had optimized for prediction accuracy alone, at the expense of inference cost, maintainability, and explainability.

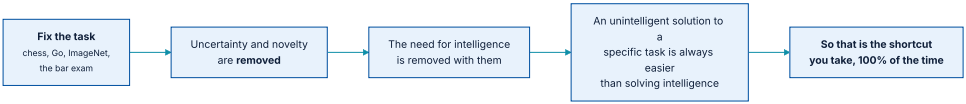
The same holds in most Kaggle competitions: winning models can **rarely, if ever, be used in production**. **Your creations are shaped by the incentives you give yourself.**

Deep Blue, and what it taught us about the mind

In 1997 Deep Blue beat Kasparov. Researchers then had to contend with the fact that it had **taught them little about human intelligence**. The A-star algorithm at its heart was not a model of the brain and could not generalize beyond similar board games.

> It turned out to be easier to build an AI that could only play chess than to build an artificial mind — **so that is the shortcut researchers took**.

Fixing the task removes the need for intelligence



The driving success metric of AI has been to solve specific tasks — from chess to Go, MNIST to ImageNet, high school maths to the bar exam.

Given near-infinite training data, even nearest-neighbour search can play video games with superhuman skill. Given near-infinite hand-written if-then-else statements, so can those — **until you make a small change to the rules, the kind a human adapts to instantly**.

SECTION 19.3.3

There is no path from skills to intelligence

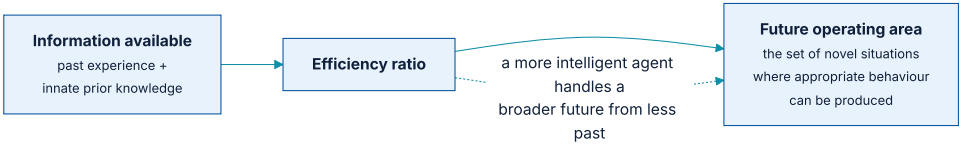
Humans can use general intelligence to acquire skill at any new task. **In reverse, there is no path from a collection of task-specific skills to general intelligence.**

By fixing the task you make it possible to describe precisely what needs to be done, and to **buy** more skill by adding data or hardcoded knowledge — without increasing generalization power at all.

> Human-like intelligence is not characterized by skill at any particular task. It is **the ability to adapt to novelty, to efficiently acquire new skills and master never-before-seen tasks**.

SECTION 19.3.4

Intelligence as an efficiency ratio



Fix the information available; measure performance on situations known to be sufficiently different from it.

To avoid cheating you must test only on tasks the system was not programmed or trained to handle — in fact, **tasks its creators could not have anticipated**.

ARC - AGI

Each task is explained by three or four **input grid / output grid** examples. You are given a new input grid and have three tries to produce the correct output before moving on.

Only unseen tasks

A game you cannot practise for. Each task has its own unique logic to be understood on the fly; memorizing strategies from past tasks does not transfer.

Controlled priors

All test takers start from **Core Knowledge priors** — the knowledge systems humans are born with. Unlike an IQ test, tasks never involve acquired knowledge such as English sentences.

SECTION 19.3.5

A 50,000× scale-up bought ten percentage points

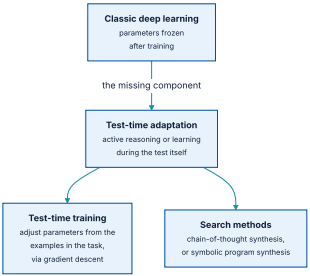
~50,000×
base LLM scale-up, GPT-2 (2019) to GPT-4.5 (2025)

0% → ~10%
their ARC-AGI score over that period

> 95%
what you, the reader, would score

Most benchmarks saturated quickly in the age of LLMs, because they can be **hacked via memorization**. ARC-AGI was designed to resist it. **Scale up 50,000× with no meaningful progress and that is a large warning sign that you need new ideas.**

The narrative shift, and what caused it



Every single top entry in ARC Prize 2024 used test-time adaptation.

TTA means the system performs **active reasoning or learning during the test itself**, using the specific problem information provided.

o3, and the first signs of fluid intelligence

COMPUTE SETTING	ARC-AGI SCORE	COST PER TASK
Moderate	**76%**	about $200
High	**88%** — above the human baseline	over **$20,000**

For the first time, a model showed **signs of genuine fluid intelligence**. ARC-AGI was one of the only benchmarks at the time giving a clear signal that a paradigm shift was underway.

SECTION 19.3.7

Is AGI solved? Not quite — and here is why

Efficiency is the point

Tens of thousands of dollars per puzzle. **Brute-forcing the solution space with enormous compute is a shortcut that makes tasks possible without requiring intelligence.** It felt more like cracking a code with a supercomputer than nimble reasoning.

Still stumped by easy tasks

o3 failed many tasks humans find **very easy** — even at the highest compute settings.

The entire point of intelligence is to achieve results with the least resources possible. In principle you could solve ARC-AGI by enumerating every possible program until one fits the demonstrations. **That would prove nothing.**

ARC-AGI-2: a benchmark that can tell degrees apart

ARC-AGI-2 keeps the exact same format with significantly harder content: **longer reasoning chains, inherently more resistant to exhaustive search**, so that computational efficiency becomes critical to success.

instant
typical human time on an ARC-AGI-1 task

~5 min
average human time on ARC-AGI-2

~0%
base LLM performance on ARC-AGI-2

Even **o3's scores plummeted into the low double digits** under reasonable compute budgets. The challenge of efficient, human-like fluid intelligence is far from solved.

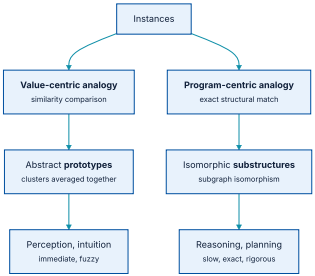
04

The missing ingredients: search and symbols

Two kinds of abstraction, and why any complete mind needs both.

SECTION 19.4.1

Two ways to compare things



Together, these two poles form the basis for all of our thoughts.

Value-centric analogy: the beetles in your backyard

These prototypes are abstract — they look like no specific beetle you have seen. Meet a new beetle and you compare it to your handful of prototypes rather than to every beetle in your memory.

> This is pretty much a description of **unsupervised machine learning** — K-means, and in general all of modern machine learning. **The ConvNet features visualized in chapter 10 were visual prototypes.**

SECTION 19.4.1

It is also most of what you do without thinking

If you can do a task without thinking about it, you are heavily relying on value-centric analogies.

It underlies pattern recognition, perception, and intuition. Watching a film and subconsciously sorting the characters into *types* is value-centric abstraction.

It is also exactly the kind of analogy-making that enables deep learning models to perform **local generalization** — which is the connection this chapter has been building toward.

Program-centric analogy: refactoring

You are **not** comparing them by how similar they look, the way you would compare two faces. You are asking whether parts of them have **exactly the same structure** — looking for a **subgraph isomorphism** between programs represented as graphs of operators.

> This is not exclusive to computer science or mathematics. It underlies reasoning, planning, and the general concept of **rigour** as opposed to intuition — any time you think about objects connected by a **discrete network of relationships**

SECTION 19.4.2 · TABLE 19.1

The two poles, side by side

VALUE-CENTRIC ABSTRACTION	PROGRAM-CENTRIC ABSTRACTION
Relates things by **distance**	Relates things by **exact structural match**
Continuous, grounded in geometry	Discrete, grounded in topology
Abstracts by *averaging* instances into prototypes	Abstracts by isolating isomorphic substructures
Underlies **perception and intuition**	Underlies **reasoning and planning**
Immediate, fuzzy, approximate	Slow, exact, rigorous
Needs a lot of experience to be reliable	**Experience efficient** — can work from two instances

SECTION 19.4.2

Nothing you do uses only one of them

And even *pure reasoning* — finding the proof of a theorem — involves a good deal of intuition. **When a mathematician puts pen to paper they already have a fuzzy vision of the direction they are going.** The discrete steps are guided by high-level intuition.

> These two poles are complementary, and **it is their interleaving that enables extreme generalization**. No mind could be complete without both.

SECTION 19.4.3

Sorting five numbers, and classifying MNIST

Sorting, with deep learning

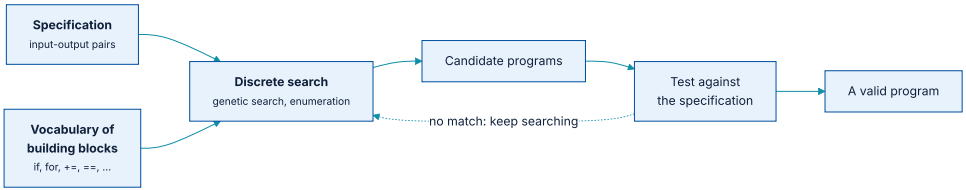
Possible with the right architecture, and **an exercise in frustration**. Massive data, occasional mistakes on new numbers, and a complete retrain to go from lists of 5 to lists of 10. A Python sort is a few lines and works on **any** list.

MNIST, with discrete reasoning

Hand-code closed-loop counts, centres of mass. After **thousands of lines** you might reach 90% test accuracy. Fitting a parametric model is simpler, uses the data better, and is far more robust.

So it is unlikely anyone will reduce reasoning to manifold interpolation, or perception to discrete reasoning. **The way forward is a unified framework incorporating both.**

Program synthesis, the missing piece



Highly reminiscent of machine learning — given input-output pairs, find a program that generalizes.

The difference: instead of learning **parameter values** inside a hardcoded program (a neural network), we **generate source code** through a discrete search process.

Machine learning against program synthesis

	MACHINE LEARNING	PROGRAM SYNTHESIS
Model	A differentiable parametric function	A graph of operators from a programming language
Engine	**Gradient descent**	**Discrete search** (e.g. genetic search)
Data	Requires a lot to be reliable	**Data efficient** — works from a couple of examples

Until 2024, AI systems capable of genuine discrete reasoning were **all hardcoded by human programmers**. In the test-time adaptation era that is finally changing — and program synthesis is the branch to watch.

Hybrids already exist, and they are hand-built

AlphaGo

Most of the intelligence on display is hardcoded by human programmers — Monte Carlo Tree Search. Learning from data happens only in specialized submodules: value and policy networks.

A Waymo self-driving car

Maintains a literal **3D model of the world**, full of assumptions hardcoded by engineers, constantly updated by deep learning perception modules — powered by Keras.

In both, the combination of human-created discrete programs and learned continuous models unlocks performance **impossible with either alone**. Today the discrete parts are painstakingly hand-coded. In the future such systems may be fully learned.

An RNN has one hardcoded for loop. What if it had more?

Now imagine a network augmented not with a single hardcoded loop and continuous-space memory, but with a large set of **programming primitives** it is free to manipulate: `if` branches, `while` statements, variable creation, disk storage, sorting operators, lists, graphs, hash tables.

> The space of programs such a network could represent would be far broader — and **some would generalize far better**. Such programs will not be differentiable end to end, so they must be generated by **a combination of discrete program search and gradient descent**

Combinatorial explosion

As specification complexity rises, or the vocabulary of primitives expands, the set of possible programs grows **much faster than merely exponentially**. So today, program synthesis generates only very short programs. *You are not going to be generating a new OS for your computer anytime soon.*

> The fix is to bring it closer to how humans write software. When you open your editor you are **not thinking about every possible program** — you have a handful of approaches in mind, cut down by understanding and past experience.

Deep learning can guide the search

A model trained on millions of successful program-generation episodes might develop **solid intuition about the path through program space** — just as an engineer has immediate intuition about the architecture of the script they are about to write.

> Human reasoning is itself heavily guided by value-centric abstraction — by pattern recognition and intuition. **The same should be true of program synthesis.** Expect increasing research interest over the next 10 to 20 years.

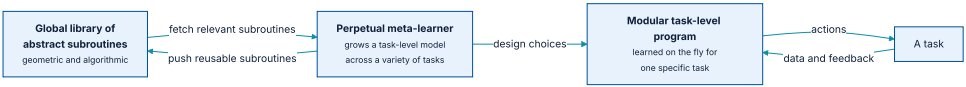
SECTION 19.4.6

What LLMs are missing, stated precisely

They are in fact **entirely incapable of function composition**, as investigated by Dziri et al. And the functions they learn are not sufficiently abstract or modular to be recombined in the first place.

> Remember the low accuracy on adding large integers? **You would not want to build your next codebase on top of such brittle functions.**

A global library, and a perpetual meta-learner



Subroutines may be geometric (pretrained deep learning modules) or algorithmic (closer to the libraries engineers use today).

Think of software development: once an engineer solves HTTP queries in Python, they package it as a reusable library accessible to anyone on the planet. **Any single problem encountered would only need to be solved once** — making such systems constantly self-improving.

SECTION 19.4.7

The long-term vision, in four points

1. **Models will be more like programs**, going far beyond continuous geometric transformations — closer to the abstract mental models humans maintain, and capable of stronger generalization.
2. **They will blend algorithmic modules** providing formal reasoning, search, and abstraction **with geometric modules** providing intuition and pattern recognition. AlphaGo and self-driving cars are an early, hand-built example.
3. **They will be grown automatically** rather than hardcoded, from modular parts in a global library evolved across thousands of previous tasks — frequent problem-solving patterns turned into reusable subroutines.
4. **The search over combinations will be discrete** — program synthesis — but heavily guided by a form of program-space intuition provided by deep learning.

WHAT TO CARRY INTO PRACTICE

Four things this chapter should change about your work

Ask about familiarity, not difficulty

When scoping an LLM deployment, the question is whether the task resembles something in web text — **not whether it is hard**.

Test with perturbations

Change names, numbers, and phrasing that do not change the problem. **If the answer changes, you have measured memorization.**

Choose the metric carefully

The shortcut rule is not a research curiosity. Optimize one number and you will get it, **at the expense of everything else** — as Netflix did.

Expect hybrids

The systems that work in the real world today combine learned perception with hand-built discrete logic. **Design for that combination**, not for an end-to-end network.

SUMMARY

What has to survive this chapter

1. **A deep learning model is a static interpolative database.** The only operation it supports is retrieval.
2. **Success tracks familiarity, not complexity.** A trivially easy but novel problem defeats every current model.
3. **Prompt engineering is search**, not communication. Its necessity is evidence against understanding.
4. **Scale has not moved any of these limits** in seven years and four orders of magnitude, because the model is unchanged.

SUMMARY

What has to survive this chapter (2 of 2)

1. **Local, broad, and extreme generalization** are different things. Deep learning does the first, and only for anticipated factors of variation.
2. **Fixing the task removes the need for intelligence** — and an unintelligent solution is always the easier one, so it is the one you get.
3. **Intelligence is an efficiency ratio** between information available and future operating area — which is what ARC-AGI measures.
4. **Test-time adaptation was the 2024 shift**: every top ARC Prize entry used it, and o3 crossed the human baseline — at $20,000 a task.
5. **Two poles of abstraction**: value-centric for perception, program-centric for reasoning. Deep learning has one of them.
6. **The way forward is a blend** — program synthesis for discrete structure, deep learning to guide the search.

PAPER
Chollet, On the Measure of Intelligence (2019)

SITE
ARC Prize

NEXT
Chapter 20 — Conclusions